

RinkConnect — the model layer as it stands

A description of everything under `lib/models/`: the live set the app runs on, and the unwired `new_models/` rewrite, contrasted. Read from the working tree on 5 September 2026.

1 — The live set: `lib/models/*.dart`

Eleven files. Hand-written, const constructors, no serialization of any kind, and almost every one carries its design reasoning in a doc comment pointing at an ADR. This is the set the running demo uses — `ClubStore` holds them all in one immutable snapshot and every derived read is a method computed over that snapshot rather than a fetch. Nine files are load-bearing; two are dead.

Family `family.dart`

A household's ongoing relationship with the club. Created once, persists across seasons, and holds **nothing** that resets annually — hours owed, dues and the ledger all live on the season membership instead. A family that stops renewing simply has no membership for the current season; it is never deleted.

```
class Family {
  final String id;
  final String name;          // "The Cheung Family"

  String get shortName;      // "Cheung" - strips "The " / " Family"
}
```

Person `person.dart`

Someone inside a household — a parent or a skater. Logins attach to a person; obligations do not. A person works a slot, and their approved hours credit their *family*'s season membership.

```
class Person {
  final String id;
  final String familyId;
  final String name;
  final bool  eligible;    // may this person's work count at all
  final bool  isSkater;    // shown differently from the adults

  String get firstName;
}
```

- The `eligible` / obligation split is the whole point: eligibility is a property of the **person** (a skater under the club's age threshold cannot earn hours), while the obligation is a property of the **season membership**.

SeasonMembership `season_membership.dart`

A family's registration for one season. Everything that resets annually lives here. These are what the board counts — “102 members last season, 77 registered for 2026–2027”.

```
class SeasonMembership {
  final String id;
  final String familyId;
  final String season;      // "2026-2027"
  final double hoursOwed;   // typed by the coordinator, never derived
  final bool   duesPaid;   // maintained by hand; no payment processing

  SeasonMembership copyWith({double? hoursOwed, bool? duesPaid});
}
```

- `hoursOwed` lives per family rather than as a club-wide constant so the coordinator can vary it — a mid-season joiner owes less, a hardship waiver is zero — with no schema change.
- `duesPaid` is independent of the hours obligation: the obligation starts at registration, not at payment, so an unpaid family still owes hours.

Event `event.dart`

Something the club runs that needs volunteers. Owns its slots outright as a list rather than referencing them.

```
class Event {
  final String id;
  final String name;
  final String body;
  final DateTime startDate;      // date and times split across
  final DateTime startTime;     //   three separate DateTimes
  final DateTime endTime;
  final String location;
  final String who;             // free text, "Open to all members"
  final String? badge;
  final List<VolunteerSlot> volunteerSlots;

  int   get slotCount;          // derived, not stored
  double get totalHours;
  String get whenLong;         // "Sat, Aug 8 - 8:00 AM - 11:00 AM"
  String get whenShort;        // "Sat Aug 8 - 8:00 AM"
}
```

- An earlier `numOfVolunteers` field was removed: it was a second representation of the same fact and could disagree with the slot list. The count is now derived.
- The date-formatting getters put presentation concerns on the model.

VolunteerSlot `volunteer_slot.dart`

One shift on an event: a role, a time range, and what it is worth. Volunteer need is expressed as **fixed slots** rather than an open hours total, because the coordinator's real problem is coverage — a named person at the check-in desk at 9am — not an hours tally. Ten parents each pledging one hour satisfies a total while leaving the door unstaffed.

```
class VolunteerSlot {
  final String id;
  final String label;          // the role
  final String timeRange;      // a DISPLAY STRING, not real times
  final double hours;          // what the slot is worth

  String get hoursLabel;       // "2" rather than "2.0"; "1.5" kept as-is
}
```

- hours is the default credited on approval, not a hard cap — the coordinator can approve fewer for someone who left early.
- The slot itself carries no notion of who claimed it; that lives on Signup.

Signup + SignupState `signup.dart, signup_state.dart`

One person claiming one volunteer slot, and where that claim sits in its lifecycle. The signup is made *by a person* but credits *their family's* season membership.

```
class Signup {
  final String id;
  final String slotId;
  final String eventId;
  final String personId;
  final SignupState state;
  final double? approvedHours; // null until approval; may differ from slot

  Signup copyWith({SignupState? state, double? approvedHours});
}

enum SignupState {
  pledged,      // slot claimed, event hasn't happened
  submitted,    // volunteer reports having worked it; awaiting coordinator
  approved,     // coordinator confirmed - this, and only this, is credit
}

extension SignupStateLabel on SignupState {
  String get label; // "Signed up" / "Awaiting review" / "Approved"
  bool get countsTowardBalance; // true only for approved
}
```

- These are three genuinely different numbers and conflating them breaks the fee calculation. A balance computed from pledges would let a family claim twenty hours of slots, attend none, and appear to owe nothing.
- countsTowardBalance is the single chokepoint that enforces that rule.

HoursBalance hours_balance.dart

A family's hours position for one season — the number the whole app exists to produce. A pure derived value object, computed from a season membership's owed figure plus that family's signups. Nothing stores it.

```
class HoursBalance {
  final double owed;      // set by the coordinator on the membership
  final double approved;  // confirmed - the only figure that discharges
  final double pending;   // pledged or submitted: SHOWN, never counted

  static const zero;
  double get remaining;   // (owed - approved), floored at 0
  double get progress;    // approved / owed, clamped to 0..1
  bool get complete;      // approved >= owed
  double feeAt(double ratePerHour); // remaining * rate
}
```

- Pending is surfaced separately so a volunteer can see work in flight without it counting as credit.
- The rate is a **parameter** to feeAt, not stored on the model. The fee is computed and handed to the treasurer; RinkConnect never collects it.

UserRole user_role.dart

Which view of the club the signed-in person sees. The demo's stand-in for auth — the header toggle swaps viewpoint by changing currentPersonId on the store.

```
enum UserRole { coordinator, parent }

extension UserRoleLabel on UserRole {
  String get label; // "Coordinator" / "Parent"
}
```

- Replaced an older DashboardVariant.board: every family owes volunteer hours, not only the board, so parent is the default view.

Dead prototype leftovers post.dart, profile.dart

Unreferenced by anything. News carries a hydrated author and a comment count; Profile duplicates the role concept with its own enum.

```
class News {
  final String id;
  final String authorId;
  final String title;
  final String body;
  final DateTime? publishedAt;
  final DateTime createdAt;
  final Profile? author;
  final int commentCount;
  bool get isPublished;
}

class Profile {
  final String id;
  final String firstname;
  final String lastname;
  final String email;
  final ProfileRole role; // parent |
  final DateTime createdAt; // coordinator
}

enum ProfileRole { parent, coordinator }
```

2 — The rewrite: lib/models/new_models/*.dart

Seven files, wired to nothing. Uniformly machine-generated in the Dart Data Class house style: an // ignore_for_file: public_member_api_docs, sort_constructors_first header, non-const constructors, and a full complement of copyWith / toMap / fromMap / toJson / fromJson / toString / == / hashCode on every class. No doc comments anywhere. The shape is relational rather than object-graph: every table hangs off a clubId, there are no owned collections, and relationships are plain foreign keys. The field lists below omit the generated boilerplate.

Club `club.dart`

The tenant root. Has no counterpart in the live set, which is single-club throughout.

```
class Club {  
  final String id;  
  final String clubName;  
  final String clubAddress;  
  final DateTime createdAt;  
}
```

Coordinator `coordinator.dart`

A coordinator as its own entity, scoped to a club and disjoint from Family. In the live set the same idea is an enum value on the signed-in person.

```
class Coordinator {  
  final String id;  
  final String clubId;  
  final String firstName;  
  final String lastName;  
  final String email;  
  final String createdAt; // String here; DateTime on Club  
}
```

Family `new_models/family.dart`

Scoped to a club, and carrying a person's name rather than a household's. There is no Person in this set at all — family and individual are one row.

```
class Family {  
  final String id;  
  final String clubId;  
  final String firstName;  
  final String lastName;  
}
```

Membership `membership.dart`

The counterpart to SeasonMembership. The season becomes an explicit date range, and the per-hour fee rate is stored on the row.

```
class Membership {  
  final String id;  
  final String familyId;  
  final String volunteeringRecordId; // singular FK on the membership  
  final DateTime startDate; // replaces the "2026-2027" string  
  final DateTime endDate;  
  final double feePerHour; // rate stored, not passed in  
  final double hoursOwed;  
  final bool isPaid; // was duesPaid  
}
```

Event `new_models/event.dart`

Same three-DateTime treatment of date and time as the live model, now scoped to a club and holding no slot list — slots point back at the event instead.

```
class Event {
  final String id;
  final String clubId;
  final String name;
  final String body;
  final String eventLocation;
  final DateTime eventStartDate;
  final DateTime eventStartTime;
  final DateTime eventEndTime;
}
```

- who and badge are gone, as are all the formatting getters.

EventSlot `event_slot.dart`

The counterpart to VolunteerSlot. Gains real start and end times instead of a display string, and carries the claiming family directly on the slot.

```
class EventSlot {
  final String id;
  final String eventId;
  final String familyId; // who claimed it - non-nullable
  final String jobDescription; // was label
  final String hoursRequired; // typed String, not double
  final DateTime startTime; // real times, not a "9-11am" string
  final DateTime endTime;
}
```

VolunteerRecord `volunteer_record.dart`

The counterpart to Signup: a record of work done against a slot, with the approving coordinator named on the row. The three-state lifecycle collapses to a boolean.

```
class VolunteerRecord {
  final String id;
  final String familyId; // the family who participated
  final String eventSlotId; // the slot participated in
  final String coordinatorId; // the coordinator who approved
  final double hoursWorked;
  final DateTime dateWorked;
  final bool isApproved; // replaces pledged/submitted/approved
}
```

3 — Where the two sets diverge

Read left to right: what the running app does today, and what the rewrite would do instead.

Concern	lib/models (live)	new_models (unwired)
Tenancy	none – single club, implicit	Club entity; clubId on Family, Coordinator, Event
The individual	Person, with familyId, eligible, isSkater	no Person – Family carries firstName / lastName
Eligibility	Person.eligible, Person.isSkater	no representation
Coordinator	UserRole enum on the signed-in person	Coordinator table, disjoint from Family

Concern	lib/models (live)	new_models (unwired)
Family name	one field: "The Cheung Family"	firstName + lastName
Season	season: "2026-2027" string	startDate / endDate range
Dues flag	duesPaid	isPaid
Fee rate	argument to HoursBalance.feeAt(rate)	feePerHour stored on Membership
Slot to event	Event owns List<VolunteerSlot>	EventSlot.eventId foreign key
Slot role text	label	jobDescription
Slot hours	double hours	String hoursRequired
Slot times	timeRange display string	startTime / endTime as DateTime
Who claimed a slot	Signup.personId (a person)	EventSlot.familyId, on the slot itself
Claim lifecycle	SignupState: pledged / submitted / approved	bool isApproved
Hours credited	Signup.approvedHours, nullable until approval	VolunteerRecord.hoursWorked, required
Approver identity	not recorded	VolunteerRecord.coordinatorId
Membership to work	derived by scanning signups	Membership.volunteeringRecordId, a single FK
Balance	HoursBalance value object with getters	no counterpart
Serialization	none	toMap / fromMap / toJson everywhere, camelCase keys
Immutability	const constructors throughout	non-const; copyWith on every class
Documentation	doc comments citing the governing ADR	none

Two entities exist in one set and not the other: Club and Coordinator are new, while Person, HoursBalance and UserRole have no counterpart in the rewrite. Everything else is a rename, a retype, or a change of where a relationship is recorded.